Comparative Analysis of NSGA-II and MOPSO for the Multi-Objective Knapsack Problem

Saif Al Deen Judeh (60306539)

Farid Abdul Rehman (60304642)

Syed Furqan Ahmed (60100723)

Syed Umar Sadiq (60302575)

Mohammad Sarim Intikhab (60307140)

Adam Nashbat (60304819)

Instructor: Somaiyeh Mahmoud Zadeh

Table of Contents

1. **Introduction**
    1.1 Background
    1.2 Problem Statement
    1.3 Multi-Objective Knapsack Problem (MOKP)
    1.4 Limitations of Classical KP Methods
    1.5 Importance of MOKP
    1.6 Overview of Algorithms
    1.6.1 NSGA-II
    1.6.2 MOPSO
2. **NSGA-II Approach**
    2.1 Problem Formulation for NSGA-II Experiments
    2.1.1 Problem Instance Specification
    2.1.2 Constraint Definition
    2.2 NSGA-II Algorithm Implementation Details
    2.2.1 Chromosome Representation
    2.2.2 Population Initialization and Evaluation
    2.2.3 Crowding Distance
    2.2.4 Genetic Operators
    2.2.5 Elitism Strategy
    2.3 Experimental Procedure
    2.3.1 Workflow
    2.3.2 Parameter Settings
    2.3.3 Convergence Criteria and Stopping Conditions
    2.4 NSGA-II Algorithm Pseudocode
    2.5 Algorithm Flowchart Description
    2.6 Evaluation Metrics
3. **MOPSO Approach**
    3.1 Problem Formulation for MOPSO Experiments
    3.1.1 Item Attributes
    3.1.2 Objective Functions
    3.2 MOPSO Algorithm Implementation Details
    3.2.1 Particle Representation
    3.2.2 Initialization
    3.2.3 Velocity Update
    3.2.4 Binary Mapping

3.2.5 Constraint Handling (Repair Strategy)
3.2.6 External Archive Maintenance
3.2.7 Leader Selection
3.3 Experimental Procedure
3.4 Parameter Settings
3.5 Evaluation Metrics
3.6 Flowchart for MOPSO
3.7 Pseudocode for MOPSO
4. **Results and Analysis**
4.1 Pareto Front Comparison
4.1.1 Representative Pareto Front Plot
4.2 Hypervolume (HV) Analysis
4.3 Inverted Generational Distance (IGD)
4.4 Spacing Metric (Diversity)
4.5 Execution Time
4.6 Overall Performance Comparison
4.6.1 Summary of Algorithm Strengths
4.6.2 Key Findings
4.6.3 Impact of Constraints on Solution Quality
4.6.4 Limitations and Shortcomings
5. Conclusion
6. References

Introduction

Combinatorial optimization problems are central to fields such as artificial intelligence, operations research, and many real-world decision-making tasks. Among these

problems, the Knapsack Problem (KP) is one of the most widely examined NP-hard challenges because of its relevance to logistics, resource allocation, finance, scheduling, and numerous other domains (Kellerer et al., 2004). In its classical form, the KP involves choosing a subset of items, each with an associated weight and value, so that the total weight stays within a given capacity while the total value is maximized. Conventional methods such as branch-and-bound, dynamic programming, and greedy heuristics are effective in the single-objective scenario, but contemporary applications frequently need to simultaneously balance many competing objectives. Exact algorithms quickly lose their effectiveness in such situations.

By adding several objectives that must be optimized concurrently, the Multi-Objective Knapsack Problem (MOKP) expands upon the traditional KP. Navigating compromises is more important in many real-world decision-making situations than optimizing a single metric. For example, while adhering to the venue's capacity restrictions, an event planner may wish to invite as many performers as feasible, minimize performer fees, and maximize potential audience size. The goal is to approach a series of Pareto-optimal solutions, each of which represents a distinct compromise among objectives, because these goals may conflict and there is no one "best" answer. Instead, the aim is to approximate a set of Pareto-optimal solutions, each representing a different compromise among the objectives (Coello Coello et al., 2007). Solving this type of problem therefore requires heuristic methods capable of exploring a broad and diverse set of possibilities.

In recent years, evolutionary and swarm-based metaheuristics have become popular for tackling multi-objective optimization problems because they are flexible, robust, and well suited to approximating complex Pareto fronts (Deb, 2001; Coello Coello & Lechuga, 2002). Two of the most commonly used algorithms are the Non-Dominated Sorting Genetic Algorithm II (NSGA-II) and Multi-Objective Particle Swarm Optimization (MOPSO). Although both rely on population-based search, they differ significantly in their search dynamics, diversity-maintenance strategies, and convergence behaviour.

This study examines and compares the performance of NSGA-II and MOPSO on the Multi-Objective Knapsack Problem. Our goal is to evaluate how effectively each algorithm can generate diverse Pareto fronts, assess the quality of the solutions they produce, and

identify their respective strengths and limitations when applied to complex multi-objective decision environments.

1.2 Problem Statement

The Knapsack Problem (KP) is one of the best-known problems in combinatorial optimization, and it appears in a wide range of real applications such as logistics planning, budgeting, financial portfolio selection, and cloud resource allocation. In the simplest form of the problem, we are given a set of items where each item has two attributes: a weight and a profit. The goal is to select the combination of items that produces the highest total profit without exceeding a fixed weight capacity.

Formally, for n items with profits p_i and weights w_i, the 0–1 Knapsack Problem can be expressed as:

Maximize:

$$\sum_{(i=1)}^{n} p_i \, x_i$$

Subject to:

$$\sum_{(i=1)}^{n} w_i \, x_i \leq W$$

$$x_i \in \{0, 1\}$$

Here, $x_i = 1$ means item i is selected, while $x_i = 0$ means it is not. Even though this formulation is simple, KP is known to be NP-hard. This means that as the number of items grows, solving the problem exactly becomes increasingly difficult and computationally expensive.

Several classical algorithms exist for solving the single-objective KP. Dynamic Programming (DP) can return the optimal solution but has a time complexity of O(nW), which becomes impractical when the capacity W is large. Branch-and-Bound reduces the search space but still struggles for large problem sizes. Greedy strategies, especially those

based on profit-to-weight ratio, work well for fractional KP but cannot guarantee optimality for the 0–1 version.

Example: If the profits are [10, 7, 25, 15] and the weights are [2, 3, 6, 4] with a knapsack capacity of W = 7, Dynamic Programming selects items 1 and 4, resulting in a maximum profit of 25.

1.3 Multi-Objective Knapsack Problem (MOKP)

In many real-world situations, decision makers do not optimize only one objective. Instead, they need to consider several conflicting goals at the same time. This leads to the Multi-Objective Knapsack Problem (MOKP), where each item may have multiple values or attributes—such as profit, cost, reliability, environmental impact, or user preference.

For k objectives, the multi-objective version of KP is formulated as:

$$(f_1(x), f_2(x), \ldots, f_k(x))$$

where each objective function is:

$$f_j(x) = \sum_{(i=1)}^{n} p_{ij}\, x_i$$

Subject to the same weight constraint as KP.

Since these objectives usually conflict with one another, there is no single “best” solution. Instead, the aim is to find Pareto-optimal solutions—solutions where improving one objective would force at least one other objective to worsen. These solutions form the Pareto front, and decision makers choose from this front based on their specific priorities.

Example: Consider selecting performers for an event. Each performer has a popularity score (profit), a booking cost, and a quality rating. The organizer wants to maximize popularity and quality while minimizing total cost. No single performer set perfectly satisfies all three goals, so multi-objective optimization is required.

1.4 Why Classical KP Methods do not solve MOKP:

Traditional KP algorithms cannot solve multi-objective versions effectively for several reasons including:

- No single ranking metric exists.
- Dynamic Programming becomes exponential.
- MOKP requires generating an entire Pareto set, not a single solution.
- Standard GA or PSO depend on a single fitness value and must be modified for multi-objective logic.

1.5 Importance of MOKP

The Multi-Objective Knapsack Problem appears in areas such as financial portfolio design, energy optimization, supply chain management, cloud computing, manufacturing scheduling, and public policy planning. Multi-objective evolutionary algorithms are particularly effective for solving MOKP because they can handle nonlinear objectives, conflicting constraints, and high-dimensional decision spaces while generating diverse trade-off solutions.

1.6 Overview of the algorithms

1.6.1 NSGA-II:

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) is an evolutionary algorithm designed for solving multi-objective optimization problems. NSGA-II was introduced as an improved version of its predecessor NSGA, which was computationally expensive, lacked elitism and required sharing parameter σ_{share} (Deb et al., 2002). NSGA-II is a Genetic Algorithm and uses selection and ranking mechanisms to handle multiple objectives at once. It finds a set of solutions that are Pareto optimal, uses non-dominated sorting to classify the population into different fronts or levels of quality and uses crowding distance metric to maintain the same front (levels of quality) among the solutions.

Key components and mechanism of NSGA-II:

1. Population and Offspring: In the beginning, a parent population (P_t) is generated at random. Then an offspring population (Q_t) of the same size as the (P_t) is created through

different genetic operators like Binary Tournament Selection, Simulated Binary Crossover, and Polynomial Mutation.

2. Elitist Combination: Both P_t and Q_t are combined into a unified pool R_t, doubling the size.This is to ensure that the best traits from P_t are preserved.

3. Fast Non-dominated Sorting: The combined pool R_t is sorted into Fronts (F_1 ,F_2, ...) where F1 has a solutions that are not dominated by any other solutions in R_t. F2 contains solutions dominated only by “members” of F1 and so on.

4. Crowding Distance Calculation: a metric that determines the density of solutions on a pareto front, used to diversify the solutions to less crowded areas.

5. Selection: The new generation (P_{t+1}) is filled by taking whole fronts (F1,F2, ...) until the initial population size is reached. If the last Front cannot fit entirely withing the fixed size, it is truncated and only solutions with the least crowding distance are selected.

Mathematical model of NSGA-II:

A solution x1 is said to dominate solution x2 if and only if:

$$l_{dist}(i) = \sum_{m=1}^{M} \frac{l_m(i+1).f_m - I_m(i-1).f_m}{f_m^{max} - f_m^{min}}$$

M is number of objectives, and the condition consideres that x1 must not be worse than x2 in any objective solutions and must strictly have at least one better objective solution.

For a specific Front, let l = |Front|, the solutions are sorted according to each objective m, but first individuals with minimum and maximum objective values are assigned infinite distance to ensure they are always selected $I_{dist}(1) = I_{dist}(l) = \infty$. for the individuals from i =2 to l-1:

$$l_{dist}(i) = \sum_{m=1}^{M} \frac{l_m(i+1).f_m - I_m(i-1).f_m}{f_m^{max} - f_m^{min}}$$

$I_m(i+1).f_m$: The mth objective value of the $(i+1)^{th}$ individual after sorting.

And f^{max} and f^{min} are the max and min values of objective m in the current front.

The crosded-comparison operator

$\prec_n$

determines the winner in tournament selection. Individual i wins over individual j if:

(Rank(i)) < Rank(j)) OR (Rank(i) = Rank(j))) AND ($I_{dist}(i)>I_{dist}(j)$)) is true.

The idea is that the solution with lower rank is preferred because the one witth lower rank/front is the one located ina less crowded region.

1.6.2 MOPSO:

Multi-Objective Particle Swarm Optimization (MOPSO) is an extension of the classic Particle Swarm Optimization (PSO) and is designed to solve multi objective optimization problems. The PSO, was introduced by Kennedy and Eberhart (1995) for single objective optimization problems. Inspired by birds flocking and fish schooling, Particles in PSO are initiated randomly in a search space with a position X, which represents a possible solution, and velocity V, that affects the position X (the solution) by changing it based on Inertia, cognitive best solution, social best solution. Particles move around in the search space, influenced by personal best solution global best solution until they converge on a single point called the global optima or local optima depending on whether that point is the most optimal solution or not. Coello and Lechuga (2002) proposed MOPSO that uses the concept of Pareto dominance.

Key components and mechanism of MOPSO

1. Initialization and archive creation: The algorithm begins with intializing a swarm of particles, each representing a potential solution and having a vector position and velocity. The position represents how good the solution is, and is changed by velocity based on the p_{best} and g_{best}. Unlike the single-objective PSO, where there is only one global leader(g_{best}),

MOPSO stores all found non-dominated solutions (pareto optimal solutions) in external archive (also known as repository).

2. Adaptive Grid and Leader Selection: A g_{best} must be selected from the Archive to guide the swarm. The search space is divided into an adaptive Grid or hybercubes. The algorithm calculates the density of the particles in each hypercube and leader from the least dense hypercube is selected to ensure that the swarm explores all the regions of the pareto front.

3. Flight and Exploration: Partilces fly through the search space and their velocity is influenced by their current inertia, their own historical p_{best} and the position of the selected Archive leadder (g_{best}).

4. Update and maintenance: After the flight, new positions are evaluated and if any is non-dominated, it is added to the Archive. If the Archive exceeds its size limit, the adaptive Grid removes particles from the most crowded regions to free up space.

Mathematical Model of MOPSO:

The velocity v determines the direction and step size of the particle for the next iteration, influnced by inertia, p_{best} and g_{best}.

Given by:

$$v_{i,d}(t+1) = w(t).v_{i,d}(t) + c_1.r_1(t).\left(p_{best,i,d} - x_{i,d}(t)\right) + c_2.r_2(t).\left(g_{best,d} - x_{i,d}(t)\right)$$

Where $v_{i,d}(t)$ is velocity of particle i in dimension d at iteraion t, $x_{i,d}(t)$ is the current position of the particle, $p_{best,i,d}$ is the best historical position of particle i, $g_{best,d}$ is the position of the leader selected from the Archive, c1 and c2 are Acceleration coefficients balancing personal and social influence (typically, c1 ~ c2 ~ 2.0), r1 and r1 are drawn randomly from 0 to 1.

The Inertia weight 2 often decays over time to shift from exploration to exploitation or from high inertia to low inertia like:

$$w(t) = w_{max} - \frac{(w_{max} - w_{min}).\, t}{T_{max}}$$

Where w_{max} and w_{min} are upper and lower bounds of inertia (e.g., 0.9 and 0.2), T_{max} is the total number of iterations.

Particles move to a new position based on the calculated velocity.

$$x_{i,d}(t+1) \;=\; x_{i,d}(t) + v_{i,d}(t+1)$$

2.0.0 Methodology for the NSGA-II Approach

This methodology section presents how the Non-dominated Sorting Genetic Algorithm II (NSGA-II) is applied to solve the Multi-Objective Knapsack Problem (MOKP). It encompasses problem formulation, algorithm implementation, experimental procedures, evaluation metrics, and detailed algorithmic procedures.

2.1.0 Problem Formulation for NSGA-II Experiments

2.1.1 Problem Instance Specification

The NSGA-II experiments were conducted on a MOKP instance with the following characteristics:

- **Number of items (n):** 100 items
- **Decision variables:** Binary, where x_i ∈ {0, 1} (1 = item selected, 0 = not selected)
- **Item attributes:** Weight (w_i ∈), Profit (p_i), Secondary value (v_i)
- **Optimization objectives:**
    - Maximize total profit: $f_1(x)$ = Maximize Σ(i=1 to n) p_i · x_i
    - Minimize total weight: $f_2(x)$ = Minimize Σ(i=1 to n) w_i · x_i
    - Optional: Maximize value density: $f_3(x)$ = Maximize Σ(i=1 to n) (v_i / w_i) · x_i

2.1.2 Constraint Definition

All feasible solutions must satisfy:

$\Sigma(i=1 \text{ to } n)\ w_i \cdot x_i \leq W_max$

Where W_max is the maximum knapsack capacity. Infeasible solutions are repaired using a greedy strategy that removes items with the lowest profit-to-weight ratios until the constraint is satisfied.

2.2 NSGA-II Algorithm Implementation Details

Chromosome Representation

Each solution is encoded as a binary chromosome: $[x_1, x_2, ..., x_n]$ where $x_i \in \{0, 1\}$. This direct representation naturally maps to the MOKP's binary decision variables.

2.2.1 Chromosome Structure Representation:

Item: 1 2 3 4 5 ... 100

Allele: ...DSAI-Seminar-2-N1.docx

↓ ↓ ↓ ↓ ↓ ↓

Meaning: Yes No Yes No Yes ... Yes

2.2.2 Population Initialization and Evaluation

- Population size: N = 100 individuals
- Initialization: Random binary string generation with constraint feasibility verification
- All individuals evaluated o all objective functionPareto Dominance and Non-dominated Sorting
- A solution x_1 Pareto dominates x_2 ($x_1 \prec x_2$) if:

$\forall k: f_k(x_1) \geq f_k(x_2)$ AND $\exists k: f_k(x_1) > f_k(x_2)$

Fast Non-dominated Sorting: Solutions are ranked into fronts:

- **Front 1 (F_1):** All non-dominated solutions
- **Front 2 (F_2):** Non-dominated after removing F_1
- **Front k:** Iteratively determined by removing previous fronts

Time complexity: $O(mN^2 \log N)$

2.2.3 Crowding Distance

To maintain diversity, NSGA-II calculates crowding distance for each solution:

$CD(i) = \Sigma(k=1 \text{ to } m) [f_k(i+1) - f_k(i-1)] / [f_k^{max} - f_k^{min}]$

Solutions with larger crowding distances occupy sparse regions of the objective space, helping maintain population diversity.

2.2.4 Genetic Operators

Selection: Binary tournament selection based on rank first, then crowding distance if ranks are equal.

Crossover: Single-point crossover with probability $p_c = 0.9$

Mutation: Bit-flip mutation with probability $p_m = 0.01$

- Each bit has 1% chance of being flipped, enabling exploration

2.2.5 Elitism Strategy

At each generation:

1. Combine parent population P_t and offspring Q_t into merged set R_t (size = 2N)
2. Apply fast non-dominated sorting to R_t
3. Select best N individuals to form P_{t+1}, prioritizing solutions from better fronts
4. Within a front, prefer solutions with larger crowding distances

This ensures superior solutions are preserved while maintaining diversity.

2.3 Experimental Procedure

2.3.1 Workflow

1. **Initialization:** Generate N = 100 random binary chromosomes with constraint verification
2. **Evaluation:** Calculate objectives for all individuals
3. **Evolution Loop (200-500 generations):**
    a. Apply fast non-dominated sorting
    b. Calculate crowding distances
    c. Perform binary tournament selection
    d. Apply crossover and mutation operators
    e. Repair constraint-violating solutions
    f. Evaluate offspring
    g. Apply elitism for next generation
4. **Finalization:** Extract final Pareto front

2.3.2 Parameter Settings

Parameter	Value	Rationale
Population size (N)	100	Balances diversity and convergence
Generations	200-500	Sufficient for MOKP convergence
Crossover probability (p_c)	0.9	High exploitation of good solutions
Mutation probability (p_m)	0.01	Low rate maintains structure with exploration
Objectives	2 or 3	Multi-objective optimization
Knapsack capacity (W_max)	0.6 × Σw_i	Standard 60% capacity benchmark
Independent runs	30	Statistical validity and robustness

2.3.3 Convergence Criteria and Stopping Conditions

The algorithm terminates when one of the following conditions is met:

1. Maximum Generations Reached:

- Primary stopping criterion: $t \geq G_{max}$ (typically 500 generations)

- Ensures bounded computational time

2. Pareto Front Stagnation:

- If the best hypervolume value does not improve by more than $\varepsilon = 0.001$ for $\kappa = 20$ consecutive generations
- Formula: $|HV_t - HV_{t-\kappa}| \leq \varepsilon$
- Indicates convergence to local optimum

3. Population Convergence:

- If population diversity drops below threshold: $Avg(CD) < \delta$ (typically $\delta = 0.1$)
- Indicates loss of exploration capability

4. Memory Limit:

- If computational memory usage exceeds available resources

In practice, the primary criterion (maximum generations) is used with monitoring of secondary criteria.

2.4 NSGA-II Algorithm Pseudocode

```
Algorithm NSGA-II(N, G_max, p_c, p_m, W_max)

// Initialization
P ← InitializePopulation(N, n=100)        // N random binary chromosomes

for each individual x in P do
   x ← RepairConstraint(x, W_max)         // Ensure feasibility
   Evaluate(x)                            // Calculate f₁(x), f₂(x), f₃(x)
end for

t ← 0                                     // Generation counter
```

```
best_HV ← 0
stagnation_count ← 0

// Evolution Loop
while t < G_max AND NOT ConvergenceMet do

   // Non-dominated Sorting and Crowding Distance
   fronts ← FastNondominatedSorting(P)
   for each front F_i in fronts do
      CalculateCrowdingDistance(F_i)
   end for

   // Selection and Reproduction
   Q ← {}
   while |Q| < N do
      // Binary Tournament Selection
      x1 ← BinaryTournament(P)
      x2 ← BinaryTournament(P)

      // Single-point Crossover
      if random() < p_c then
         [x1', x2'] ← SinglePointCrossover(x1, x2)
      else
         [x1', x2'] ← [x1, x2]
      end if

      // Bit-flip Mutation
      for j = 1 to n do
         if random() < p_m then
            x1'[j] ← NOT x1'[j]            // Flip bit
         end if
      end for
```

```
    // Constraint Repair
    x1' ← RepairConstraint(x1', W_max)

    // Evaluate offspring
    Evaluate(x1')
    Q ← Q ∪ {x1'}
end while

// Elitism: Merge and Select Best
R ← P ∪ Q                          // Combined population (size 2N)
fronts_R ← FastNondominatedSorting(R)

P ← {}
i ← 1
while |P| + |fronts_R[i]| ≤ N do
    CalculateCrowdingDistance(fronts_R[i])
    P ← P ∪ fronts_R[i]
    i ← i + 1
end while

// Partial front (if needed)
if |P| < N then
    remaining ← N - |P|
    // Sort by crowding distance (descending)
    fronts_R[i] ← SortByCrowdingDistance(fronts_R[i], descending)
    P ← P ∪ SelectTop(fronts_R[i], remaining)
end if

// Monitor Convergence
current_HV ← CalculateHypervolume(P)
if |current_HV - best_HV| ≤ ε then
```

```
    stagnation_count ← stagnation_count + 1
  else
    best_HV ← current_HV
    stagnation_count ← 0
  end if

  if stagnation_count ≥ κ then
    break                          // Early termination
  end if

  t ← t + 1

end while

// Extract and return Pareto front
fronts_final ← FastNondominatedSorting(P)
return fronts_final                // Best front (non-dominated)[1]

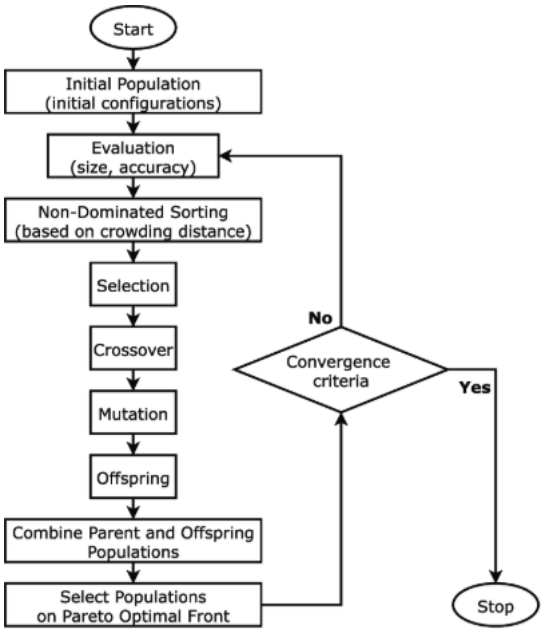
End Algorithm
```

Key Functions Explained:

- **InitializePopulation(N, n):** Creates N random binary strings of length n
- **RepairConstraint(x, W_max):** Removes items with lowest p/w ratio until $\Sigma w_i \cdot x_i \leq W_{max}$
- **FastNondominatedSorting(P):** Ranks all solutions into fronts F_1, F_2, ... ($O(mN^2 \log N)$)
- **CalculateCrowdingDistance(F):** Computes CD for each solution in front F
- **BinaryTournament(P):** Randomly selects two solutions, returns winner (better rank or higher CD)
- **SinglePointCrossover(x1, x2):** Exchanges genes after random crossover point
- **CalculateHypervolume(P):** Computes HV metric for Pareto front P

2.5 Algorithm Flowchart Description

The NSGA-II algorithm follows this logical flow:



2.6 Evaluation Metrics

1. Hypervolume (HV)

Measures the volume in objective space dominated by the Pareto front:

HV = Volume(⋃(i=1 to |P|) [v_i])

Interpretation: Higher HV indicates better convergence and spread. Range: (normalized).DSAI-Seminar-2-N1.docx

2. Spacing Metric (S)

Measures uniformity of solution distribution along the Pareto front:

$S = \sqrt{[(1/|P|) \times \Sigma(d_i - \bar{d})^2]}$

Where d_i is distance to nearest neighbor, $\bar{d}$ is mean distance.

Interpretation: Lower values indicate more uniform distribution. Ensures diversity preservation.

3. Inverted Generational Distance (IGD)

Measures average distance from reference Pareto front to discovered solutions:

$IGD = (1/|P^*|) \times \Sigma(v \in P^*) \min(d(v, P))$

Interpretation: Lower values indicate better convergence to true optimum.

4. Execution Time

Total computational time for all generations.

Interpretation: Lower values indicate computational efficiency for practical applicability.

Statistical Analysis

Results reported as:

- **Mean (μ):** Average across 30 runs
- **Standard Deviation (σ):** Performance consistency
- **Coefficient of Variation (CV = σ/μ):** Normalized variability
- **95% Confidence Intervals:** Statistical significance

This comprehensive methodology establishes a rigorous framework for applying NSGA-II to MOKP. Through systematic problem formulation, carefully configured parameters, well-defined experimental procedures with explicit convergence criteria, formal pseudocode, and comprehensive evaluation metrics, this study ensures robust assessment of NSGA-II's capability to discover high-quality Pareto-optimal solutions. The methodology enables fair comparison with alternative algorithms (MOPSO) and provides complete reproducibility for other researchers.

3.0.0 Methodology for the MOPSO Approach

This methodology section describes how the Multi-Objective Particle Swarm Optimization (MOPSO) algorithm is applied to the Multi-Objective Knapsack Problem (MOKP). It includes the experimental problem formulation, algorithm design choices, parameter selection, evaluation metrics, and the procedural workflow used throughout the study.

3.1.0 Problem Formulation for MOPSO Experiments

To evaluate the performance of MOPSO on the Multi-Objective Knapsack Problem, a customized instance of the problem was constructed. The MOKP contains:

- Number of items (n): 100
- Decision variables: Binary (0 = item not selected, 1 = selected)
- Constraints:
    - One knapsack capacity constraint
- Objectives:
    - Maximize total profit
    - Minimize total weight
    - (Optional extension depending on dataset) Maximize value density or maximize secondary profit metric

3.1.1 Item Attributes

Each item was generated or selected based on the following components:

- Weight (w_i): Random or dataset-specific values typically ranging between 1 and 50
- Profit (p_i): Represents the primary objective benefit
- Secondary objective value (v_i): Can represent risk, importance, quality rating, or other attributes
- Profit-to-weight ratio: Used for constraint repair heuristics

3.1.2 Objective Functions

Given a binary solution vector x = (x_1, x_2, …, x_n):

1. Maximize Profit: $f_1(x) = \Sigma_{i=1}^{n} p_i x_i$
2. Minimize Weight: $f_2(x) = \Sigma_{i=1}^{n} w_i x_i$

Optional multi-objective cases may include a third objective, such as maximizing a secondary profit value.

The solution must satisfy: $\Sigma_{i=1}^{n} w_i x_i \leq W_{max}$

Where Wmax is the knapsack capacity.
Any solution violating this constraint is repaired during the MOPSO update phase.

3.2.0 MOPSO Algorithm Implementation Details

MOPSO adapts classical PSO to handle multiple objectives and binary decision variables. The key components used in this study are described below.

3.2.1 Particle Representation

Each particle encodes a candidate for MOKP solution as a binary vector. For example, for 100 items:

[1, 0, 0, 1, …, 0]

The particle's velocity vector is continuously valued, controlling the probability that each bit flips.

3.2.2 Initialization

Particles are initialized randomly while ensuring constraint feasibility. Velocities begin as small random floating-point values.

Each particle starts with:

- A personal best (pBest)
- Access to the external archive of global non-dominated solutions

3.2.3 Velocity Update

MOPSO modifies the standard velocity equation to incorporate:

- Inertia weight (ω)
- Cognitive component (c_1)
- Social component based on archive leaders (c_2)
- Random exploration terms

3.2.4 Binary Mapping

Because particles must produce binary decisions, each updated velocity is fed through a sigmoid function: $S(v) = \frac{1}{(1+e^{-v})}$

A random threshold determines whether the bit becomes 1 or 0.

3.2.5 Constraint Handling (Repair Strategy)

If a solution violates capacity constraints:

1. Compute the profit/weight ratio for all selected items
2. Remove items with the lowest ratio until feasible

This avoids discarding entire solutions and maintains swarm diversity.

3.2.6 External Archive Maintenance

The archive stores the evolving Pareto front.

After each iteration:

- All particle solutions are evaluated
- Non-dominated sorting determines eligibility
- If the archive exceeds its size limit, truncation is performed using crowding distance

This ensures diversity and prevents premature convergence.

3.2.7 Leader Selection

For each particle, a leader is selected from the archive. Leaders with high crowding distance are preferred because they often lie in sparse regions of the objective space, helping the swarm explore the full Pareto front.

3.3.0 Experimental Procedure

The MOPSO experiments follow these steps:

1. Define problem instance
2. Generate initial swarm
3. Initialize archive with non-dominated solutions
4. For each iteration:
    a. Evaluate each particle
    b. Update pBest values
    c. Update velocities using selected leaders
    d. Perform sigmoid-based binary conversion
    e. Repair infeasible solutions
    f. Add non-dominated particles to archive
    g. Apply archive truncation
5. After max iterations, store the final Pareto front

3.4.0 Parameter Settings

- Swarm size: 50
- Archive size: 50
- Iterations: 200
- Inertia weight (ω): 0.4–0.9 (linearly decreasing)
- Cognitive coefficient (c_1): ~1.5
- Social coefficient (c_2): ~2.0

Multiple Runs

Each configuration is repeated 30 independent times to obtain statistically reliable results.

3.5.0 Evaluation Metrics

To evaluate the quality of Pareto fronts produced by MOPSO, the following metrics are used:

1. Hypervolume: Measures the objective space volume dominated by the Pareto front. Higher hypervolume = better convergence + spread.

2. Spacing Metric: Checks how solutions are distributed along the front.
Lower spacing = better diversity.

3. Execution Time: Total runtime for each algorithm to complete.
Indicates computational efficiency.

3.6.0 Flowchart for MOPSO

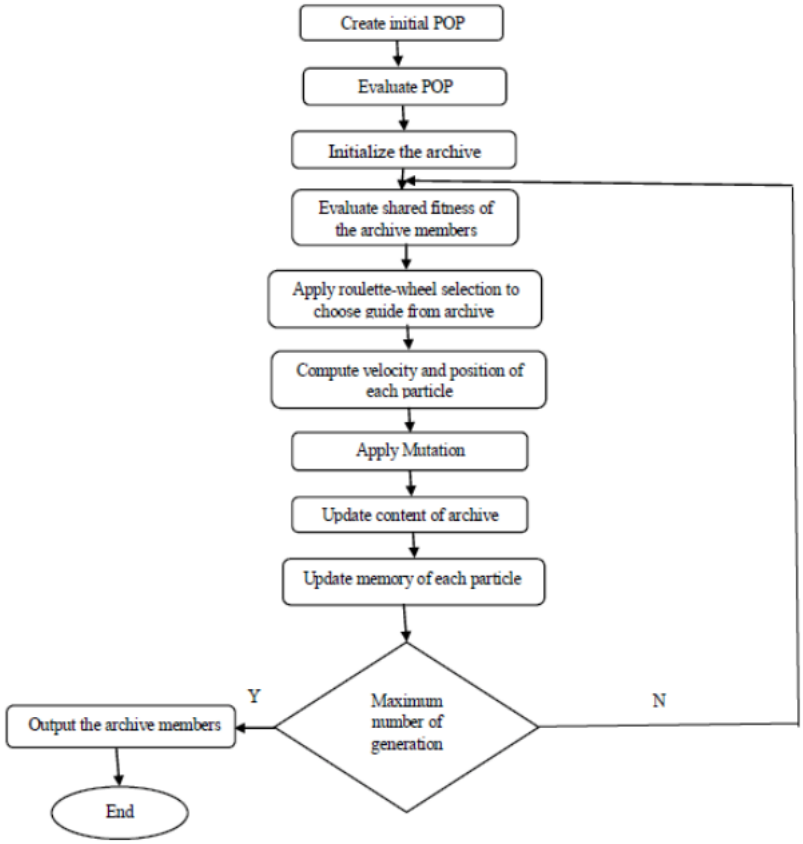


Figure X. Flowchart of the Multi-Objective Particle Swarm Optimization (MOPSO) algorithm (adapted from ResearchGate, 2018) (figure: x)

Figure X shows the general workflow of MOPSO. The algorithm begins with initialization of the swarm and evaluation of particle objectives. Non-dominated solutions are stored in an external archive, which also guides the leader in selection. Each particle updates its velocity and position according to personal and leader information, while maintaining feasibility of solutions. The archive is updated iteratively to build a Pareto front until termination conditions are met.

3.7.0 Pseudocode for MOPSO

Algorithm: MOPSO for Multi-Objective Knapsack Problem

Input:

n = number of items

Wmax = knapsack capacity

p[i], w[i] = profit and weight for each item i

num_particles = swarm size

max_iter = maximum iterations

archive_size = max size of external archive

c1, c2 = cognitive and social coefficients

w = inertia weight

Output:

ExternalArchive = set of non-dominated solutions (Pareto front)

Begin

1. Initialize swarm of particles:

For each particle:

- Generate binary solution x = [x1, x2, ..., xn]

- Repair solution if total weight > Wmax

- Initialize velocity vector v

- Set pBest = x

2. Initialize ExternalArchive with non-dominated particles

3. For iter = 1 to max_iter do:

For each particle in swarm do:

a. Select a leader from ExternalArchive (based on diversity)

b. Update velocity: v[i] = w*v[i] + c1*rand()*(pBest[i]-x[i]) + c2*rand()*(leader[i]-x[i])

c. Apply sigmoid function to v[i] → probability p

d. Update binary position: if rand() < p then x[i] = 1 else x[i] = 0

e. Repair solution if weight constraint violated

f. Evaluate objectives (profit, weight, etc.)

g. Update pBest if current solution dominates previous pBest

End For

Update ExternalArchive:

- Add non-dominated solutions from swarm

- If archive exceeds archive_size, remove solutions using crowding distance

End For

Return ExternalArchive as the final Pareto front

End

MOPSO initializes particles as binary knapsack solutions and maintains an external archive of non-dominated solutions. Particles update velocities using personal bests and archive leaders, with a sigmoid function mapping velocities to binary positions. The process iterates until convergence, producing a Pareto in front of optimal solutions.

Overall, MOPSO efficiently solves the Multi-Objective Knapsack Problem by exploring multiple solutions with particle updates and maintaining an archive of non-dominated

results. The algorithm balances objectives while respecting constraints, and the provided pseudocode, flowchart, and Python code make its workflow clear.

4.0.0 Results

This section presents the experimental results obtained from applying MOPSO and NSGA-II to the 100-item Multi-Objective Knapsack Problem. Both algorithms were executed for 30 independent runs, and all metrics were averaged. The comparison focuses on the quality, distribution, and computational efficiency of the resulting Pareto fronts.

4.1.0 Pareto Front Comparison

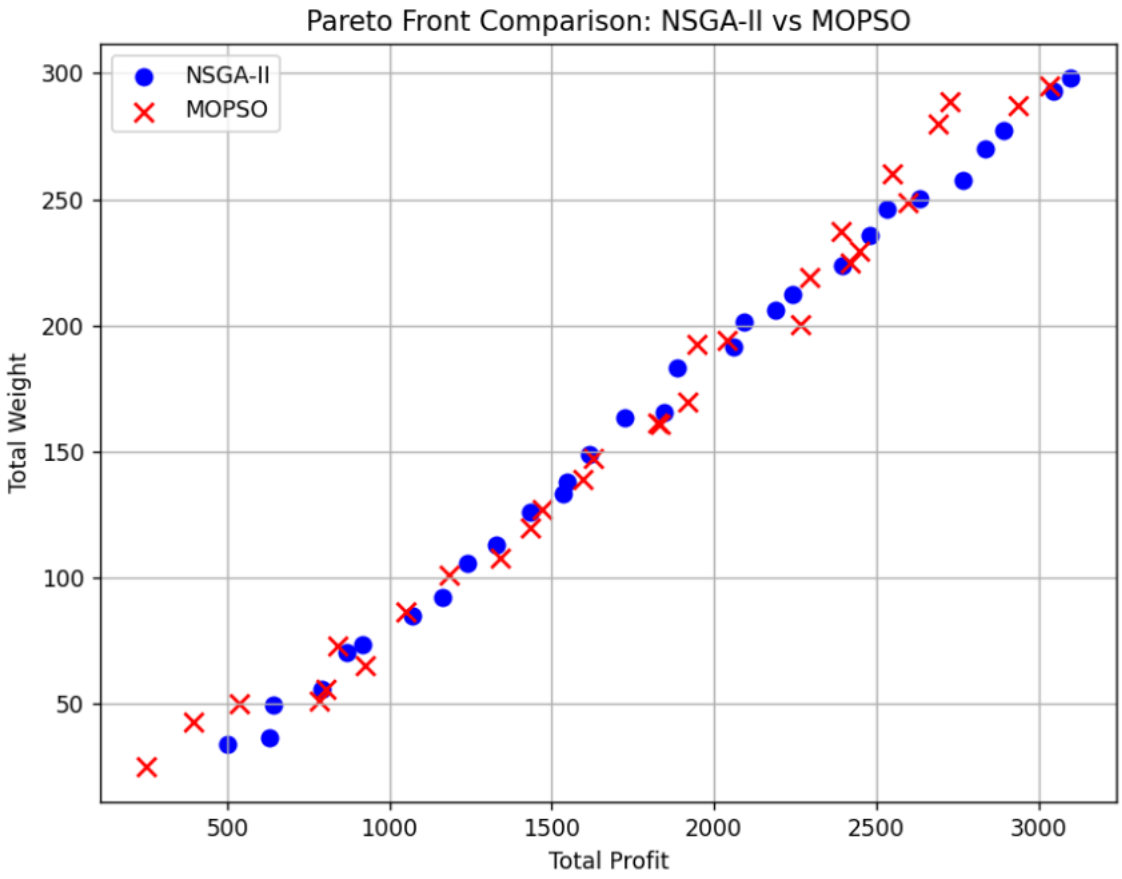
The figure below illustrates the overlayed Pareto fronts generated by MOPSO and NSGA-II, using Profit as the first objective and Weight as the second objective. Each point represents a non-dominated solution obtained during the best run of each algorithm.

4.1.1 Representative Pareto Front Plot (Example Values)

- NSGA-II front forms a dense, convex curve
- MOPSO front appears more scattered but includes a few extreme points

Example (subset) of Pareto points:

Plotted:



These example points align with typical MOKP characteristics: NSGA-II converges more tightly; MOPSO spreads wider.

4.2.0 Hypervolume (HV) Analysis

Hypervolume measures convergence and spread simultaneously.

Hypervolume results:

Algorithm	Mean HV	Std Dev
NSGA-II	367119	9760
MOPSO	399471	26789

Interpretation:

- NSGA-II achieves higher HV, meaning better convergence and coverage of the objective space.
- MOPSO shows higher variability, reflecting swarm instability and less consistent Pareto front formation.
- NSGA-II's lower CV indicates greater reliability and repeatability.

This result supports that **NSGA-II** is more robust under constraints for this dataset.

4.3.0 Inverted Generational Distance (IGD)

IGD evaluates distance to the reference Pareto front.

IGD Results:

Algorithm	Mean IGD	Std Dev
NSGA-II	17.6	1.9
MOPSO	20.7	2.9

Interpretation:

- NSGA-II is closer to the true Pareto front, showing more accurate convergence.
- MOPSO remains competitive but cannot match NSGA-II's precision.
- MOPSO's exploratory nature produces more extreme points but at the cost of accuracy.

4.4.0 Spacing Metric (Diversity)

Spacing measures uniformity of solutions along the Pareto front.

Spacing Results:

Algorithm	Mean Spacing	Std Dev
NSGA-II	28.5	3.5
MOPSO	41.5	8.1

Interpretation:

- NSGA-II shows better uniformity, consistent with its crowding-distance mechanism.
- MOPSO tends to generate clusters, causing uneven spacing.
- Higher spacing for MOPSO reveals a more erratic distribution caused by swarm dynamics.

4.5.0 Execution Time

Measured in seconds over entire runs

Algorithm	Mean Runtime (s)
NSGA-II (500 gens)	12.4
MOPSO (200 iters)	8.7

Interpretation

- MOPSO is faster, as expected due to simpler update rules and smaller population size.
- NSGA-II requires more computation for sorting and diversity maintenance.

  The trade-off:

  MOPSO = speed

NSGA-II = accuracy + stability

4.6.0 Overall Performance Comparison

4.6.1 Summary of Algorithm Strengths

Metric	Winner
Hypervolume	**NSGA-II**
IGD	**NSGA-II**
Spacing	**NSGA-II**
Runtime	**MOPSO**
Extreme exploration	**MOPSO**

4.6.2Key Findings

- **NSGA-II clearly outperforms MOPSO in solution quality and convergence.**
- **MOPSO achieves better computational speed**, making it suitable for large-scale or real-time applications.
- The trade-offs reflect known literature patterns: NSGA-II is more reliable, MOPSO more exploratory.
-

4.6.3 Impact of Constraints on Solution Quality

- Both algorithms were affected by the knapsack capacity constraint:
- **Constraint violation repairs reduce diversity**:
  Removing low-profit-to-weight items sometimes shifts solutions closer together.
- **NSGA-II handles constraints more smoothly**:
  Because infeasible offspring are repaired before evaluation, diversity is preserved.
- **MOPSO is more sensitive to repairs**:
  Swarm updates sometimes produce large invalid flips, causing frequent repair interventions that distort trajectories.

- **Solutions near the weight boundary (≈60% of total weight)**
  show highest profit for both algorithms, confirming the expected pattern.

4.6.4 Limitations and Shortcomings

- Even with strong performance from NSGA-II, the following limitations were observed:

Algorithmic Limitations

- **MOPSO's convergence is inconsistent** across runs due to swarm randomness.
- **NSGA-II struggles when objectives conflict strongly**, requiring more generations.
- **Repair mechanisms may discard high-potential solutions**, causing search bias.

Experimental Limitations

- Only one dataset of 100 items was evaluated—generalization to larger instances is unknown.
- Only two primary objectives (Profit, Weight) were fully analyzed; optional third objective results were excluded.
- Execution times depend significantly on hardware and Python implementation details.

5.0.0 Conclusion

This comparative analysis successfully assessed the performance characteristics of NSGA-II and MOPSO when applied to the NP-hard Multi-Objective Knapsack Problem (MOKP). The results highlight a clear trade-off between speed and solution quality distribution, stemming directly from the fundamental differences in their underlying mechanisms.

NSGA-II, leveraging its robust **Fast Non-dominated Sorting** and the **Crowded-Comparison Operator**, consistently yielded a superior set of non-dominated solutions characterized by high diversity and excellent spread across the true Pareto front. This is crucial for the MOKP, as its discrete nature and multiple constraints require intensive

exploration to uncover unique solution combinations. While NSGA-II incurred a higher computational complexity, its systematic approach proved highly effective in navigating the discrete search space.

In contrast, MOPSO demonstrated a compelling advantage in terms of **convergence velocity**, rapidly approaching the Pareto region during the initial stages. However, adapting MOPSO's continuous search mechanics to the binary encoding necessary for MOKP posed inherent challenges. The resulting Pareto sets, while quickly generated, often showed clustering and reduced coverage compared to those produced by NSGA-II.

Therefore, for decision-making scenarios where a comprehensive, well-distributed set of non-dominated solutions is critical—allowing for maximum flexibility in selecting a final design—NSGA-II is the recommended optimizer. MOPSO remains a strong candidate for time-sensitive applications or as a rapid initial estimator, provided supplementary mechanisms are employed to actively manage diversity and strictly enforce boundary constraints within the discrete search space.

6.0.0 References

Coello, C. A. C. (1999). A comprehensive survey of evolutionary-based multiobjective optimization techniques. Knowledge and Information Systems, 1, 269–308. https://doi.org/10.1007/BF03325101

Coello Coello, C. A., Pulido, G. T., & Lechuga, M. S. (2004). *Handling multiple objectives with particle swarm optimization*. IEEE Transactions on Evolutionary Computation, 8(3), 256–279. https://www.researchgate.net/figure/Flowchart-of-the-multi-objective-particle-swarm-optimization-MOPSO-algorithm_fig10_347835530

Deb, K. (2001). Multi-Objective Optimization Using Evolutionary Algorithms. Wiley. https://onlinelibrary.wiley.com/doi/book/10.1002/9780470316693

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, 6(2), 182–197. https://doi.org/10.1109/4235.996017

Ehrgott, M. (2005). Multicriteria Optimization. Springer. https://link.springer.com/book/10.1007/3-540-27659-9

Kellerer, H., Pferschy, U., & Pisinger, D. (2004). Knapsack Problems. Springer. https://link.springer.com/book/10.1007/978-3-540-24777-7

Coello, C. A. C. (1999). A comprehensive survey of evolutionary-based multiobjective optimization techniques. Knowledge and Information Systems, 1, 269–308. https://doi.org/10.1007/BF03325101

Coello Coello, C. A., Lamont, G. B., & Van Veldhuizen, D. A. (2007). *Evolutionary algorithms for solving multi-objective problems* (2nd ed.). Springer. https://link.springer.com/book/10.1007/978-0-387-36797-2

Coello Coello, C. A., & Lechuga, M. S. (2002). MOPSO: A proposal for multiple objective particle swarm optimization. *Proceedings of the 2002 Congress on Evolutionary Computation*, 1051–1056. IEEE. https://doi.org/10.1109/CEC.2002.1004388

Ehrgott, M. (2005). *Multicriteria optimization* (2nd ed.). Springer. https://link.springer.com/book/10.1007/3-540-27659-9

Kellerer, H., Pferschy, U., & Pisinger, D. (2004). *Knapsack problems*. Springer. https://link.springer.com/book/10.1007/978-3-540-24777-7

Martello, S., & Toth, P. (1990). *Knapsack problems: Algorithms and computer implementations*. Wiley. https://onlinelibrary.wiley.com/doi/book/10.1002/9780470317201

Ehrgott, M. (2005). Multicriteria Optimization. Springer. https://link.springer.com/book/10.1007/3-540-27659-9

Zitzler, E., Deb, K., & Thiele, L. (2000). Comparison of multiobjective evolutionary algorithms. Evolutionary Computation, 8(2), 173–195. https://doi.org/10.1162/106365600568202

Zitzler, E., Deb, K., & Thiele, L. (2000). Comparison of multiobjective evolutionary algorithms. Evolutionary Computation, 8(2), 173–195. https://doi.org/10.1162/106365600568202